1. PartnerProof

This is the one I would investigate first. Corgi is building enterprise partnerships where insurance needs to work inside another company's product. I would not build another API testing tool. I would build a way to prove that a partner can safely complete the **full insurance transaction** before the integration goes live.

The product: Insurance Integration Certification Engine

Suppose a new TMS partner connects to Corgi. The basic question is not "does the API return 200?" The real question is: **can this partner safely complete a Corgi insurance transaction when things go wrong?**

Example

```
Customer
  ↓
Eligibility → Quote → Accept → Payment
  ↓
Bind → Policy issued → COI → Webhook
```

What I would test

I would deliberately test cases such as timeout, duplicate requests, a webhook arriving twice or late, payment succeeding while the bind response is lost, quote expiry, partner retries, partial outages, and events arriving out of order.

What I want the system to catch

> **For example, the test could force a partner to retry a bind request. FAIL - Duplicate bind can occur under retry scenario. The problem is found before the integration is in production.**

What I would actually build

I would build the test environment, scenario runner, failure simulation, transaction-state checks, webhook checks, and a simple certification report showing exactly what passed and failed.

Why I find it technically interesting

This is distributed-systems engineering applied to insurance. The hard part is not checking endpoints; it is keeping the business transaction correct when networks fail or systems disagree.

This is a proposed direction, not a claim that Corgi does not already have something similar.

2. EvidenceGate

This is the idea I like most for my document and AI background. Corgi is already working with insurance documents and underwriting data. I would not try to replace document extraction. I would build the layer **after extraction** that makes the information reliable enough to use.

The product: Evidence reconciliation for underwriting

Imagine the same claim appears in several documents: **Claim #1842 - $42,000**
Claim #1842 - $67,000
Claim #1842 - $75,000 Instead of silently choosing one value, the system keeps the evidence, shows the conflict, and explains which value should currently be used.

Example

```
Claim #1842

$42K  - Source A
$67K  - Source B
$75K  - Source C

Current value: $75K
Reason: latest authoritative evidence
Impact: underwriting input
```

What I would test

I would look for duplicate claims, old versus new loss runs, different claim numbers for the same claim, changed reserves, conflicting policy dates, and conflicting fleet or driver information.

What I want the system to catch

> **The important part is provenance. Instead of only storing “incurred_loss = $75,000”, the system keeps where that number came from, when it was received, what document/page it came from, and what earlier observation it replaced.**

What I would actually build

I would build a versioned evidence store, conflict detection, claim matching, review workflow, and audit trail. The output would be a clean underwriting snapshot that can feed the existing workflow.

Why I find it technically interesting

This combines AI with serious backend engineering. AI can extract and match information, while deterministic rules and human review keep the final data understandable and traceable.

This is a proposed direction, not a claim that Corgi does not already have something similar.

3. BindSafe

This is the most backend-heavy idea of the three. An insurance transaction can involve quote, payment, bind, policy issuance, documents and partner webhooks. A timeout does not necessarily mean the transaction failed.

The product: A safe recovery layer for quote-to-bind transactions

A customer pays. The bind succeeds. Then the service crashes before it receives the response. The customer sees an error and clicks “Try again.” A normal system could accidentally create the transaction twice. I would build a system that can recover the original state before doing anything again.

Example

```
Payment  → SUCCESS
Bind     → UNKNOWN
Policy   → UNKNOWN

Service restarts
        ↓
Reconcile with the authoritative system
        ↓
Bind     → SUCCESS
Policy   → POL-72931
```

What I would test

I would test server crashes, network timeouts, duplicate requests, payment succeeding while the response is lost, bind succeeding while a webhook is delayed, duplicate webhooks, partner retries, and out-of-order events.

What I want the system to catch

> **The rule is simple: Do not guess. Recover the truth. If the original bind succeeded, the system should find that result instead of creating another bind.**

What I would actually build

I would build durable transaction state, idempotency keys, an event/outbox layer, recovery workers, reconciliation logic, and a small operations view for transactions that need attention.

Why I find it technically interesting

This is production reliability work: distributed systems, state management, recovery, and insurance business rules all meet in one problem.

This is a proposed direction, not a claim that Corgi does not already have something similar.